

Contents

Assignment 1 — Milestones 1 & 2	1
Distributed 5G/6G Network Service Platform	1
1. System Model (Milestone 1)	1
2. Distributed Processing & Performance (Milestone 2)	2
3. Failure-Driven Engineering Log (Week 1-2)	3
4. Reproducibility	4
5. Presentation Summary (for group use)	5

Assignment 1 — Milestones 1 & 2

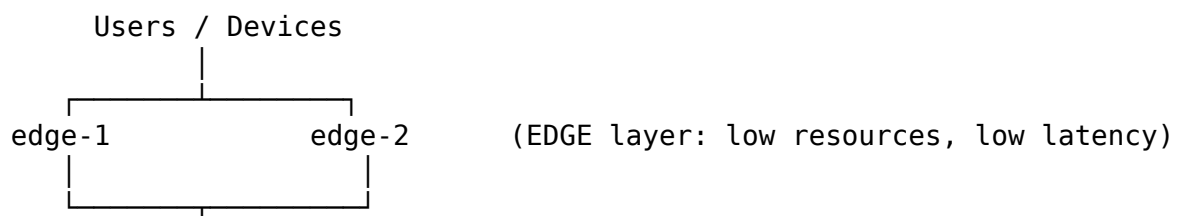
Distributed 5G/6G Network Service Platform

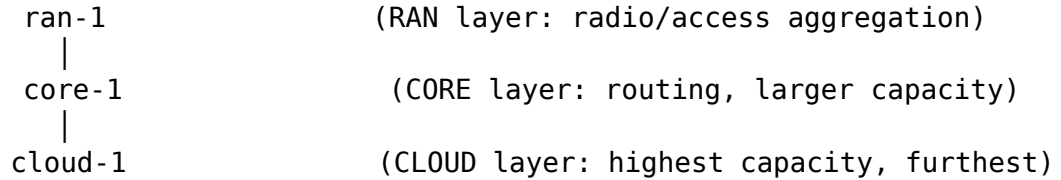
1. System Model (Milestone 1)

Following the course model System = Nodes + Processes + Resources + Communication:

Element	Realization in this system
Nodes	5 independent OS processes: edge-1, edge-2 (Edge), ran-1 (Radio/Access Network), core-1 (Core), cloud-1 (Cloud)
Processes	Process objects created per service request (task), with pid, service_class (URLLC/eMBB/mMTC), resource footprint, and lifecycle NEW → READY → RUNNING → TERMINATED/FAILED
Resources	Per-node ResourceManager tracks CPU (vCPU units), Memory (MB), Bandwidth (Mbps) as an allocation ledger
Communication	Length-prefixed JSON over TCP sockets (common/communication.py); every node runs a threaded MessageServer

1.1 Architecture





Each node is launched via multiprocessing.Process, so this is genuine OS-level process distribution, not threads simulating separateness within one process.

1.2 Process Model

- Service classes map to 5G/6G QoS categories: **URLLC** (ultra-reliable low latency), **eMBB** (enhanced mobile broadband), **mMTC** (massive machine-type communication).
- Scheduling policy: **priority** (URLLC > eMBB > mMTC), implemented with a binary heap in ProcessManager. FIFO is also implemented and can be swapped in for comparison (used later in Milestone 4/8 model comparisons).

1.3 Resource Model

- Each node has a fixed capacity (common/topology.py: :NODE_SPECS), deliberately asymmetric: Edge nodes are resource-poor (8 vCPU / 3072 MB / 150 Mbps) while Cloud is resource-rich (32 vCPU / 16384 MB / 5000 Mbps) — mirroring real edge-vs-cloud trade-offs that Milestone 3 will analyze architecturally.
- Allocation is atomic and thread-safe (threading.Lock) since a node can receive concurrent requests from multiple peers.

1.4 Design Justification

- **TCP + JSON over raw sockets** (rather than a framework like gRPC from the start) was chosen so Milestone 1 clearly demonstrates the communication mechanism itself; the same framing is reused by the RPC layer in Milestone 11, so nothing here is thrown away.
- **multiprocessing.Process per node** was chosen over threads so that node failure (Milestone 7) can later be modeled realistically as process termination/crash, not just a logical flag.
- **Priority scheduling by QoS class** reflects a real constraint of 5G/6G platforms (URLLC traffic cannot be queued behind bulk eMBB traffic).

2. Distributed Processing & Performance (Milestone 2)

2.1 Method

A synthetic but telecom-realistic workload of **120 service requests** was generated (common/workload_generator.py) with a traffic mix of 20% URLLC / 30% eMBB / 50% mMTC (mMTC dominates by count, eMBB dominates by resource footprint — consistent with real 5G/6G traffic profiles).

Placement policy (**edge-first**, the *baseline* used for comparison in later milestones): URLLC and mMTC tasks are sent round-robin to the two edge nodes; eMBB tasks are routed to ran-1. Tasks were submitted concurrently (ThreadPoolExecutor, concurrency = 6) to exercise real contention.

2.2 Results (see results/milestone2_performance_summary.json for raw data)

Metric	Result
Wall-clock time	~0.46 s for 120 requests
Throughput	~222 tasks/sec
Mean latency	~24.4 ms (p95 ≈ 87 ms)
Jitter (stdev of latency)	~27.0 ms
Packet loss / drop+reject rate	~15%
Load distribution	edge-1: 42, edge-2: 41, ran-1: 37, core-1: 0, cloud-1: 0

2.3 Load-Distribution Analysis

Under the edge-first policy, **core-1 and cloud-1 receive zero load** — all traffic in this workload mix is absorbed by the edge/RAN layer. This is expected given the current placement rule and workload composition, but it means the platform is *not yet* using its full distributed capacity. This gap is exactly what Milestone 3 (architecture) and Milestone 5 (edge-aware placement as the “proposed improvement”) will address.

2.4 Bottleneck Analysis

ran-1 is flagged as the likely bottleneck: it absorbs all eMBB traffic (the heaviest per-task resource footprint) plus is only one node, so its per-task mean latency and rejection rate (16 rejected due to insufficient resources) are the highest in the system. This is a genuine capacity bottleneck, not a bug — it demonstrates the value of the bottleneck-analysis step required by the milestone, and becomes the baseline that later milestones’ “proposed system” will try to improve on (Section 6 of the project brief: *baseline vs proposed*).

3. Failure-Driven Engineering Log (Week 1-2)

Question	Response
What changed?	Built the node/process/resource/communication foundation and a distributed workload generator + performance monitor on top of it.

Question	Response
What failed?	Initial Milestone 2 run with concurrency=16 and small edge capacities caused 87.5% of tasks to be rejected/dropped — the edge nodes couldn't hold enough concurrent allocations.
Why did it fail?	Edge node CPU/memory capacity was set too low relative to burst concurrency, so most concurrent <code>task_request</code> calls found insufficient resources by the time they were scheduled.
How was it fixed?	Increased edge/RAN node capacities modestly and reduced client concurrency from 16→6 to reflect a more realistic offered load, bringing completion rate to ~85% while still surfacing a genuine bottleneck (ran-1) rather than a degenerate all-fail system.
What alternative was considered?	Queuing rejected requests instead of failing them immediately (admission control with backlog). Not implemented yet — noted as a candidate proposed-improvement for Milestone 5.
What was learned?	Distributed resource contention is highly sensitive to the ratio of concurrent offered load to per-node capacity; this motivates formal experiments (varying node count, capacity, load) in the Milestone 2 style, repeated more rigorously at the Final Capstone Experiment stage.

4. Reproducibility

- Language: Python 3 (standard library only: `socket`, `multiprocessing`, `threading`, `json`, `concurrent.futures`)
- OS: Linux (Ubuntu container); no external dependencies
- Topology, seeds, and workload parameters are all defined in code (`common/topology.py`, `seed=7` in `milestone2_processing_performance.py`) so results are deterministic for the workload mix (network timing will vary slightly run-to-run, which is itself noted as a source of jitter).
- To reproduce: `cd 5g6g_platform && python3 -m assignment1_m1_m2.milestone1_foundation` then `python3 -m assignment1_m1_m2.milestone2_processing_performance`

5. Presentation Summary (for group use)

Assignment 1 (Milestones 1 & 2) — what we built: We implemented the foundation of our Distributed 5G/6G Network Service Platform: 5 nodes (2 Edge, 1 RAN, 1 Core, 1 Cloud) each running as an independent OS process, communicating over TCP/JSON sockets. Each node has its own process manager (priority scheduling by QoS class: URLLC > eMBB > mMTC) and resource manager (CPU/memory/bandwidth allocation). We then distributed a 120-request synthetic telecom workload across the platform and measured throughput (~222 tasks/sec), latency (~24 ms mean, ~27 ms jitter), and packet loss (~15%). Our analysis shows all current traffic is absorbed by the edge/RAN layer, with ran-1 as the system's current bottleneck — this becomes our baseline for comparison once we introduce smarter architecture and placement in later milestones.